

SOFTWARE PROJECT MANAGEMENT REPORT

Inventory Management System

A detailed academic project report for a full-stack MERN inventory solution with secure authentication, product catalog management, stock movement tracking, dashboard insights, and a soft-tone responsive user interface.

Project Type Full-stack MERN web application	Technology Stack MongoDB, Express, React, Node.js, Vite
Documentation Style SPM and SRS report in experiment format	Date 09 April 2026

Declaration and Report Note

Document Note

This report has been prepared for the Inventory Management System project in the same broad academic presentation style as the supplied sample file. It uses experiment-based organization and expanded explanatory material so the final report is substantial enough for formal submission.

The formatting is intentionally black-and-white for print readability. Diagrams are monochrome, headings are structured, and each page is explicitly separated for clean PDF generation.

Report Components

- Problem statement and SRS
- UML-style diagrams
- WBS and Gantt planning
- Software metrics and costing
- Resource allocation and team organization
- Testing, risk, feasibility, implementation, and conclusion

Table of Contents

Section	Page
Abstract and Project Summary	4
Aim	5
Theory of Software Project Management	6
Problem Statement Part I	7
Problem Statement Part II	8
Proposed System Objectives	9
SRS Introduction	10
Definitions, Acronyms, and Overview	11
Product Perspective	12
Product Functions	13
User Classes and Characteristics	14
Operating Environment	15
Design Constraints and Assumptions	16
Functional Requirements: Authentication	17
Functional Requirements: Catalog Management	18
Functional Requirements: Stock Movement	19
Functional Requirements: Dashboard and Reporting	20
Non-Functional Requirements	21
External Interface Requirements	22
Data Dictionary	23
Use Case Diagram	24
Use Case Diagram Explanation	25
Class Diagram	26
Class Diagram Explanation	27
Activity Diagram	28
Sequence Diagram	29
Component Diagram	30
Deployment Diagram	31
Authentication Module	32

Section	Page
Catalog Module	33
Stock Movement Module	34
Dashboard Module	35
Frontend Design Strategy	36
Backend Design Strategy	37
Input Design	38
Output Design	39
Aim and Theory of WBS	40
WBS Part I	41
WBS Part II	42
WBS Output and Interpretation	43
Aim and Theory of Gantt Chart	44
Gantt Schedule	45
Milestones and Dependencies	46
Schedule Risk and Control	47
Aim and Theory of Software Metrics	48
Size Metrics	49
Effort and Time Metrics	50
Productivity and Quality Metrics	51
Metric Interpretation and Output	52
Aim and Theory of Cost Estimation	53
Development Cost Estimation	54
Testing, Documentation, and Support Cost	55
Total Cost and Cost Observation	56
Aim and Theory of Resource Allocation	57
Human Resource Allocation	58
Hardware and Software Allocation	59
Time Resource Allocation and Observation	60
Aim and Theory of Team Structure	61
Team Roles and Responsibilities	62
Responsibility Matrix	63

Section	Page
Communication and Governance Plan	64
Testing Strategy	65
Representative Test Cases	66
Validation and Error Handling	67
Risk Management	68
Feasibility Analysis	69
Implementation Summary	70
Deployment Notes	71
Future Enhancements	72
Conclusion	73
References	74

Document Note

This report follows the same broad academic pattern as the sample document: experiment-wise presentation, detailed theory, problem statement, software requirements specification, UML diagrams, planning artefacts, metrics, cost analysis, resource allocation, team organization, and concluding observations.

The final report is intentionally formatted in black-and-white text for print-friendly submission. The material is expanded so the generated PDF reaches a length comparable to the sample file.

Abstract and Project Summary

Abstract

The Inventory Management System is a full-stack MERN application created to support organized product records, stock movement tracking, and dashboard-based inventory visibility. The project responds to the practical need for a centralized inventory platform in place of disconnected spreadsheets and manual registers.

The implemented application allows users to sign in, manage products, group records into categories, track stock changes, and observe operational status through a dashboard. The codebase includes a React frontend, an Express backend, and a Mongo-oriented design with a stable demo runtime for local execution.

This report documents the project in an academic style parallel to the supplied sample, covering aim, theory, problem statement, detailed SRS, diagrams, WBS, Gantt planning, metrics, costing, resource planning, team structure, testing, risk, feasibility, implementation, and conclusion.

Project Snapshot

Item	Details
Project Name	Inventory Management System
Architecture	Client-server MERN architecture
Primary Users	Admin and Staff
Core Modules	Authentication, Catalog, Stock Movement, Dashboard
Deployment Readiness	GitHub + Render deployment support prepared
Documentation Goal	Detailed SPM/SRS report matching the sample style

Aim

Aim Statement

To study, analyze, design, implement, and document an Inventory Management System and to prepare a complete software project management report in the same experiment-oriented academic style as the reference sample.

Specific Goals

1. Understand the inventory management problem and the limitations of manual systems.
2. Prepare a detailed Software Requirements Specification.
3. Model the system through UML and project planning artefacts.
4. Develop a working MERN solution with inventory and dashboard features.
5. Document the project in a print-ready SPM report of comparable length to the sample.

Theory of Software Project Management

Theory

Software Project Management is the discipline through which software work is planned, organized, monitored, and controlled. It ensures that technical work is aligned with goals, time limits, budget assumptions, and quality expectations.

A software project cannot be managed only by coding. Requirements, scheduling, design, testing, deployment preparation, documentation, and risk control all interact with one another. Good management therefore creates the conditions in which coding can actually succeed.

For an inventory system, project management is particularly relevant because the domain includes both information accuracy and process accuracy. A small defect in quantity handling can lead to visible operational mistakes, which makes disciplined design and verification essential.

This report adopts that discipline by connecting requirement engineering, design modelling, planning, cost analysis, and implementation into one coherent academic narrative.

Problem Statement Part I

Discussion

Existing System Study is an important part of the Inventory Management System because it directly affects understanding present inventory limitations. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches existing system study through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, existing system study influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Manual and spreadsheet-driven record keeping often produces duplicate or stale data.
- Warehouse staff may not have one shared live source of truth for current stock.
- Reorder decisions are delayed because alerts are not built into the process.
- Movement history is hard to trace when logs are informal or distributed.

Observation

The study of the existing environment shows a strong need for centralization, validation, and timely visibility.

Problem Statement Part II

Discussion

Operational Challenges is an important part of the Inventory Management System because it directly affects stock visibility and reliability. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches operational challenges through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, operational challenges influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Users need quick answers about available quantity and item location.
- Managers need low-stock signals before shortages become operational problems.
- Teams need movement history to investigate mismatches and accountability.
- Organizations need a more reliable estimate of inventory value.

Observation

The project therefore focuses on a system that reduces ambiguity, shortens lookup effort, and makes stock state easier to trust.

Proposed System Objectives

Primary Objectives

1. Create a centralized product and category repository.
2. Enable secure user authentication and protected access to inventory functions.
3. Support stock-in, stock-out, and adjustment operations with validation.
4. Provide dashboard insights for value, category spread, and low-stock situations.
5. Offer a calm, readable soft-tone interface for routine operations.

Expected Outcomes

- Reduced inventory confusion and duplicate record handling.
- Faster product lookup and movement logging.
- Improved visibility into reorder-sensitive stock conditions.
- A maintainable academic MERN codebase ready for further extension.

SRS Introduction

Purpose of the SRS

The Software Requirements Specification defines what the Inventory Management System must do and under which assumptions it will operate. It serves developers, testers, project reviewers, and stakeholders by giving them one structured reference point.

The SRS is especially important for this project because stock-related behavior should not be left implicit. Validation rules, user roles, and data relationships need to be stated clearly before or alongside implementation.

Scope of the SRS

The SRS covers system boundaries, user classes, functional requirements, non-functional requirements, interfaces, design assumptions, and requirement-linked diagrams for the Inventory Management System.

Definitions, Acronyms, and Overview

Term	Meaning
SRS	Software Requirements Specification
UI	User Interface
API	Application Programming Interface
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
WBS	Work Breakdown Structure
SDLC	Software Development Life Cycle

Overview

The report is organized into experiments to match academic presentation. Early pages focus on requirements and design. Middle pages focus on planning and measurement. Later pages focus on resources, testing, implementation, and project closure.

This structure mirrors the logic of real project work: understand the problem first, then plan and model the system, then estimate and organize resources, and finally review implementation and quality.

Product Perspective

Discussion

Product Perspective is an important part of the Inventory Management System because it directly affects system architecture and positioning. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches product perspective through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, product perspective influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- The system is a standalone web application using a client-server model.
- The React frontend provides interaction and display.
- The Express backend provides validation, routing, and business logic.
- MongoDB acts as the persistent data layer in standard MERN mode.

Observation

The product perspective confirms that the system is best understood as a layered web application rather than a single monolithic script.

Product Functions

Major Functions

- User registration and login
- Product creation and update
- Category-wise inventory organization
- Search and filter operations
- Stock-in, stock-out, and adjustment recording
- Low-stock and out-of-stock monitoring
- Recent movement history and inventory value display

Function Summary

These functions together create a compact but practical inventory workflow suitable for both academic demonstration and small-scale operational use.

User Classes and Characteristics

User Class	Characteristics	System Need
Admin	Higher control, responsible for overview and configuration	Access to dashboard, product updates, and movement operations
Staff	Operational user focused on routine activity	Fast stock logging and clear product lookup

Interpretation

The user model remains deliberately simple so the project stays focused. Even with only two classes, responsibility boundaries are visible and useful for design and testing.

A future enhancement could add a manager role for analytics without full administrative permissions.

Operating Environment

Aspect	Description
Platform	Web-based application
Client	Modern browser
Frontend Runtime	React and built static assets
Backend Runtime	Node.js with Express
Database	MongoDB Atlas or local Mongo instance
Deployment	Render-ready production setup

Environment Note

The operating environment is intentionally mainstream so the project remains portable, teachable, and easy to review.

Design Constraints and Assumptions

Design Constraints

- The interface must remain readable and responsive across common desktop and mobile widths.
- Authentication must protect API routes from unauthorized access.
- Invalid stock movements must be rejected before quantity corruption occurs.
- The project must remain simple enough for academic submission and explanation.

Assumptions and Dependencies

- Users provide correct stock details and movement quantities.
- The deployment environment provides Node.js support.
- MongoDB connectivity is available in normal MERN mode.
- For local demonstration, a stable demo runtime may be used when database setup is unavailable.

Functional Requirements: Authentication

Requirement ID	Requirement
FR-01	The system shall allow users to create an account using name, email, and password.
FR-02	The system shall validate user credentials during login.
FR-03	The system shall return an authenticated session token after successful login.
FR-04	The system shall protect inventory and dashboard routes from anonymous access.

Functional Requirements: Catalog Management

Requirement ID	Requirement
FR-05	The system shall allow authorized users to create product records.
FR-06	The system shall ensure SKU uniqueness.
FR-07	The system shall allow updating of existing product records.
FR-08	The system shall store category, supplier, location, quantity, and pricing data for each product.

Functional Requirements: Stock Movement

Requirement ID	Requirement
FR-09	The system shall support stock-in, stock-out, and adjustment transactions.
FR-10	The system shall recalculate quantity and stock status after each valid movement.
FR-11	The system shall reject stock-out operations when requested quantity exceeds available stock.
FR-12	The system shall store movement history including reference and note.

Functional Requirements: Dashboard and Reporting

Requirement ID	Requirement
FR-13	The system shall compute total inventory value.
FR-14	The system shall list low-stock and out-of-stock items.
FR-15	The system shall display recent stock activity in reverse chronological order.
FR-16	The system shall provide category-wise inventory distribution.

Non-Functional Requirements

Quality Requirements

- Performance: normal interactions should be completed quickly under regular workload.
- Security: credentials and protected routes must be handled securely.
- Usability: forms and tables must remain readable and manageable.
- Maintainability: backend and frontend code should remain modular.
- Portability: the application should run in modern browser and Node environments.

Interpretation

The non-functional requirements ensure that the system is not only feature-complete but also practical, maintainable, and reviewable.

External Interface Requirements

User Interface Requirements

- The login screen shall provide clear entry for email and password.
- The dashboard shall present summary cards and activity without clutter.
- The inventory screen shall provide search, filters, forms, and movement logging.
- The interface shall remain consistent across desktop and mobile layouts.

Software and Communication Interfaces

- Frontend communicates with backend using JSON over HTTP.
- Backend communicates with MongoDB using Mongoose in MERN mode.
- Render deployment serves the built frontend and backend from one service.

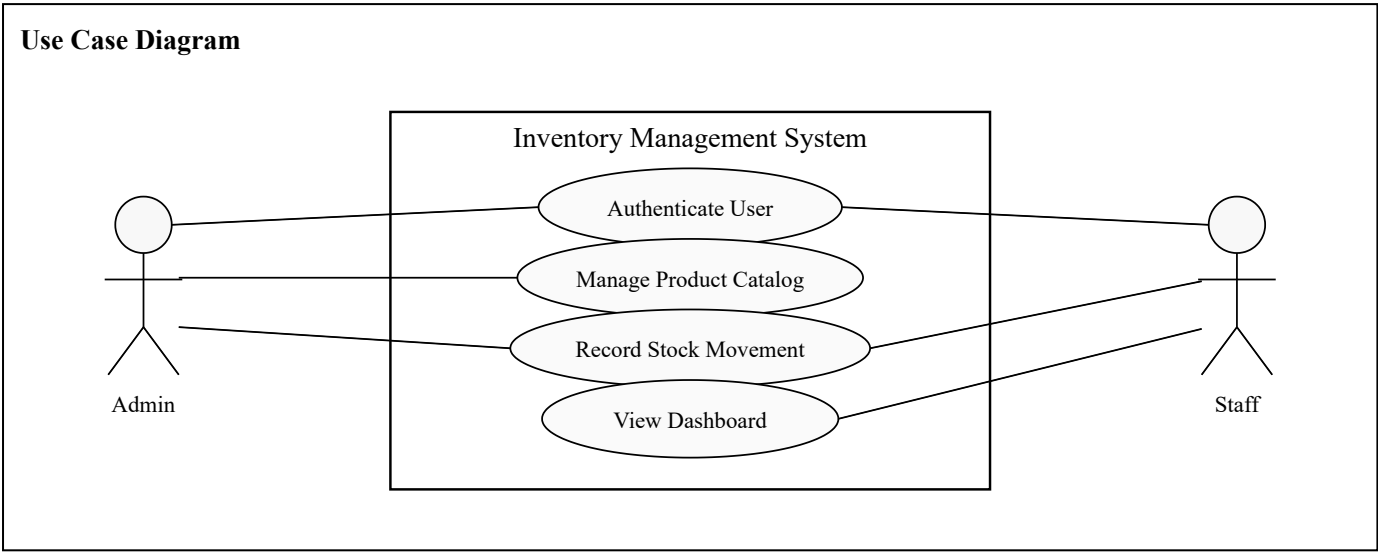
Data Dictionary

Entity	Key Attributes	Purpose
User	id, name, email, role	Access control and action ownership
Category	name, slug, accentColor	Grouping and display segmentation
Product	name, sku, quantity, reorderLevel, supplier, location	Core inventory record
StockMovement	type, quantity, previousQuantity, newQuantity	Historical stock trace

Business Rule Note

The business rules around product and movement data are intentionally strict because inventory accuracy depends on them.

Use Case Diagram



The use case diagram identifies the two main actors: Admin and Staff. Admin participates in all four central use cases, while Staff interacts mainly with authentication, stock movement, and dashboard viewing.

This diagram helps clarify functional scope at a glance and ensures that implementation remains aligned with expected user interactions.

Use Case Diagram Explanation

Discussion

Use Case Interpretation is an important part of the Inventory Management System because it directly affects actor responsibilities and system boundaries. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches use case interpretation through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, use case interpretation influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

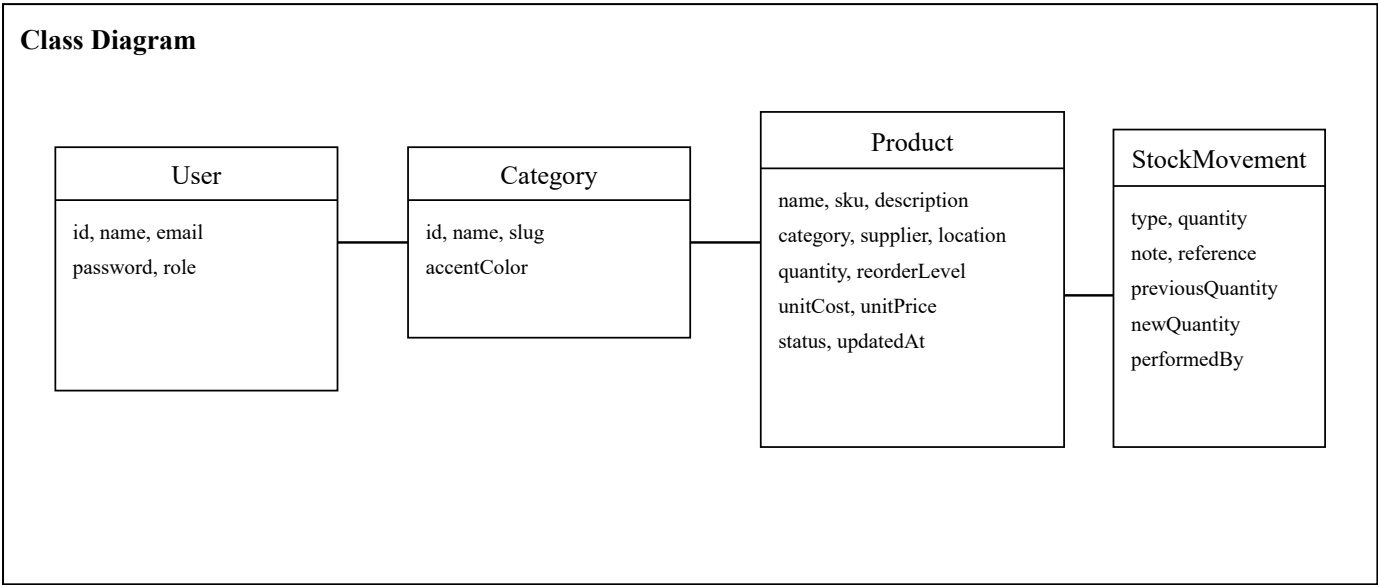
Key Points

- Admin needs control over both product structure and stock state.
- Staff needs a shorter path to operational updates and monitoring.
- The system boundary prevents actors from interacting with persistence directly.
- Each use case is intentionally broad enough to represent a real working task.

Observation

The discussion of use case interpretation shows that the project is not only about writing code. It also requires deliberate control over actor responsibilities and system boundaries so the implemented system remains correct, understandable, and suitable for academic evaluation.

Class Diagram



The class diagram captures the static structure of the application. Product is the domain center, Category supports grouping, User supports accountability, and StockMovement supports traceability.

This model guided both the backend schema design and the frontend assumption about how data objects are related when displayed in forms, tables, and summaries.

Class Diagram Explanation

Discussion

Class Relationships is an important part of the Inventory Management System because it directly affects structural clarity and maintainability. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches class relationships through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, class relationships influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

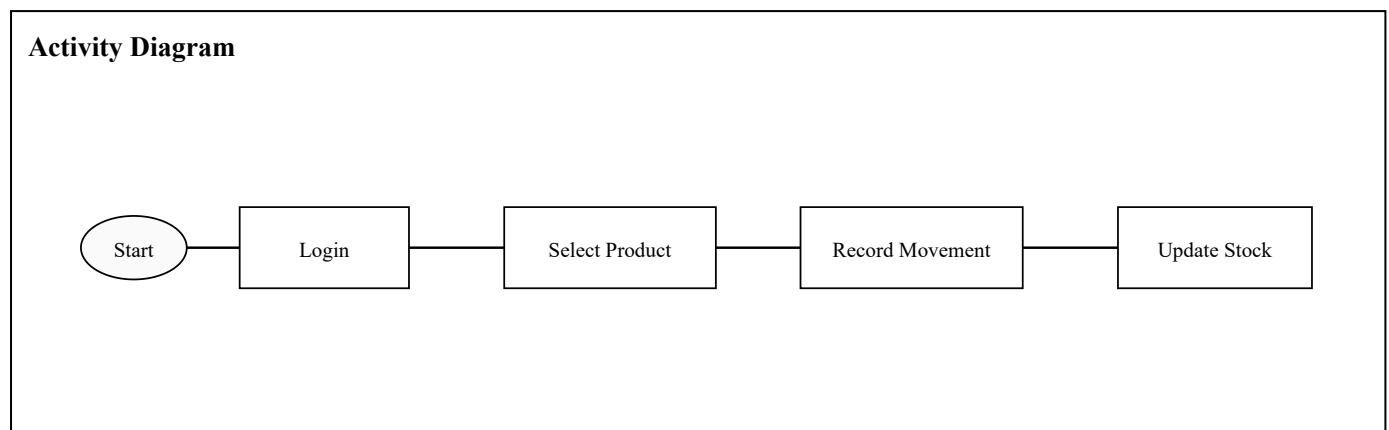
Key Points

- User-to-movement association supports responsibility tracking.
- Category-to-product association supports filtering and display grouping.
- Product-to-movement association supports change history.
- Clear entity separation improves controller and route organization.

Observation

The discussion of class relationships shows that the project is not only about writing code. It also requires deliberate control over structural clarity and maintainability so the implemented system remains correct, understandable, and suitable for academic evaluation.

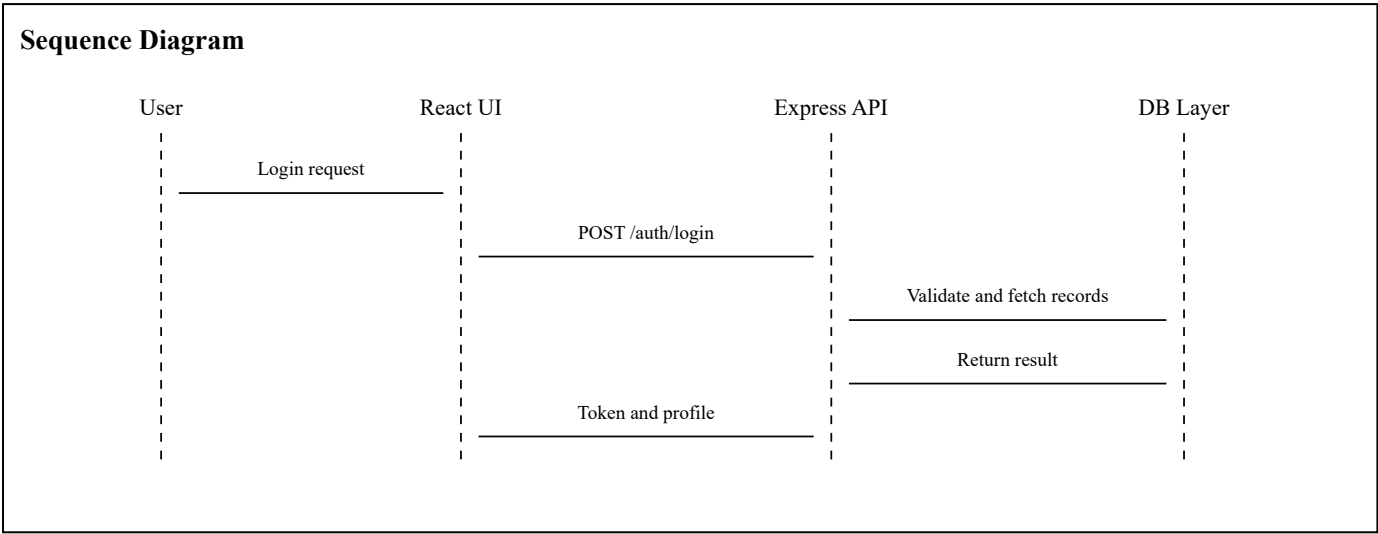
Activity Diagram



The activity diagram presents a simplified operational flow from login to stock update.

It is useful because it highlights sequence rather than structure, making warehouse-style task flow easier to explain to non-programmer reviewers.

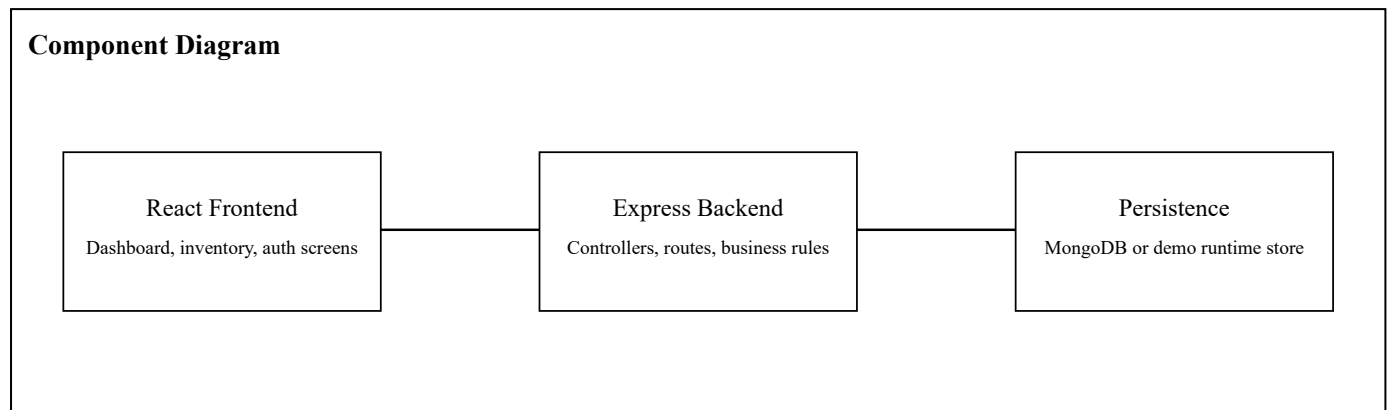
Sequence Diagram



The sequence diagram shows the order of messages between user, client, backend, and data layer.

It demonstrates that the frontend acts as an interface layer while business rules remain centralized at the backend level.

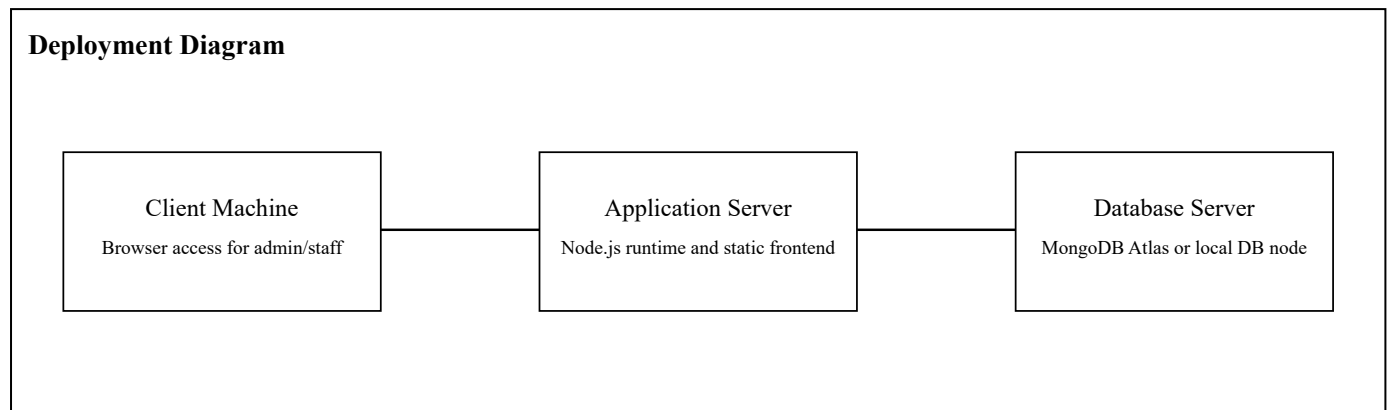
Component Diagram



The component diagram separates the project into frontend, backend, and persistence responsibilities.

This decomposition is directly reflected in the folder structure of the delivered workspace.

Deployment Diagram



The deployment diagram represents how the browser, application server, and database layer interact.

Even in academic local execution, this logical separation remains important because it informs deployment, security, and debugging decisions.

Authentication Module

Discussion

Authentication Module is an important part of the Inventory Management System because it directly affects secure access and session control. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches authentication module through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, authentication module influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Supports user registration and login.
- Protects backend routes through token-based access control.
- Separates anonymous routes from protected operational routes.
- Creates the security foundation for inventory operations.

Observation

The discussion of authentication module shows that the project is not only about writing code. It also requires deliberate control over secure access and session control so the implemented system remains correct, understandable, and suitable for academic evaluation.

Catalog Module

Discussion

Catalog Module is an important part of the Inventory Management System because it directly affects maintaining clean product information. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches catalog module through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, catalog module influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Stores SKU, supplier, location, and pricing information.
- Supports update operations without re-creating records.
- Links products with categories for organized browsing.
- Acts as the stable reference point for movement operations.

Observation

The discussion of catalog module shows that the project is not only about writing code. It also requires deliberate control over maintaining clean product information so the implemented system remains correct, understandable, and suitable for academic evaluation.

Stock Movement Module

Discussion

Stock Movement Module is an important part of the Inventory Management System because it directly affects quantity correctness and traceability. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches stock movement module through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, stock movement module influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Handles stock-in, stock-out, and adjustment transactions.
- Prevents invalid negative stock states.
- Preserves reference, note, and before/after quantity values.
- Provides the operational backbone of the inventory system.

Observation

The discussion of stock movement module shows that the project is not only about writing code. It also requires deliberate control over quantity correctness and traceability so the implemented system remains correct, understandable, and suitable for academic evaluation.

Dashboard Module

Discussion

Dashboard Module is an important part of the Inventory Management System because it directly affects summary visibility and alert-oriented insight. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches dashboard module through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, dashboard module influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Computes inventory value and status-oriented counts.
- Lists recent activity for quick operational review.
- Highlights low-stock and out-of-stock conditions.
- Supports faster managerial understanding of warehouse state.

Observation

The discussion of dashboard module shows that the project is not only about writing code. It also requires deliberate control over summary visibility and alert-oriented insight so the implemented system remains correct, understandable, and suitable for academic evaluation.

Frontend Design Strategy

Discussion

Frontend Design Strategy is an important part of the Inventory Management System because it directly affects usable and calm interaction design. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches frontend design strategy through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, frontend design strategy influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Soft-tone styling reduces harsh visual contrast during routine use.
- Cards, forms, and tables are grouped to support scan-friendly reading.
- Responsive layout allows the same interface to degrade gracefully on smaller screens.
- Clear hierarchy improves confidence when entering stock-sensitive data.

Observation

The discussion of frontend design strategy shows that the project is not only about writing code. It also requires deliberate control over usable and calm interaction design so the implemented system remains correct, understandable, and suitable for academic evaluation.

Backend Design Strategy

Discussion

Backend Design Strategy is an important part of the Inventory Management System because it directly affects validation, modularity, and predictable api behavior. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches backend design strategy through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, backend design strategy influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Routes, controllers, models, and utilities are separated into focused files.
- Validation logic is kept near the business operation it protects.
- Response payloads are structured to support predictable frontend use.
- The architecture remains simple enough for academic explanation.

Observation

The discussion of backend design strategy shows that the project is not only about writing code. It also requires deliberate control over validation, modularity, and predictable api behavior so the implemented system remains correct, understandable, and suitable for academic evaluation.

Input Design

Discussion

Input Design is an important part of the Inventory Management System because it directly affects accuracy during data entry. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches input design through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, input design influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Form fields are grouped logically to reduce confusion.
- Numeric fields are used for quantity and pricing values.
- Dropdowns are used for categories and movement types where appropriate.
- Validation messages guide the user before invalid states are accepted.

Observation

The discussion of input design shows that the project is not only about writing code. It also requires deliberate control over accuracy during data entry so the implemented system remains correct, understandable, and suitable for academic evaluation.

Output Design

Discussion

Output Design is an important part of the Inventory Management System because it directly affects clear information presentation. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches output design through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, output design influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Dashboard cards summarize the most important operational indicators.
- Tables provide structured product and movement views.
- Status pills help users identify stock conditions quickly.
- Recent activity panels support short-term operational recall.

Observation

The discussion of output design shows that the project is not only about writing code. It also requires deliberate control over clear information presentation so the implemented system remains correct, understandable, and suitable for academic evaluation.

Aim and Theory of WBS

Aim

To study the principles of project decomposition and to prepare a Work Breakdown Structure for the Inventory Management System.

Theory

A Work Breakdown Structure divides the total scope of a project into smaller and manageable pieces. It helps the team understand what must be done, who may own each task, and how progress should be tracked.

For software projects, WBS is especially useful because hidden work such as testing, integration, and documentation is often underestimated unless it is explicitly represented.

WBS Part I

WBS ID	Task	Subtask	Output
1.0	Requirement Analysis	Problem study and feature identification	Requirement notes
1.1	SRS Preparation	Formal requirement documentation	SRS chapter
2.0	System Design	Architecture and data model creation	Design package
2.1	UML Modelling	Use case, class, and activity views	Diagram set

WBS Part II

WBS ID	Task	Subtask	Output
3.0	Backend Development	Routes, controllers, models, auth	API implementation
4.0	Frontend Development	Pages, forms, dashboard, routing	React implementation
5.0	Integration and Testing	API linkage and flow validation	Working app
6.0	Documentation	SPM report and PDF output	Submission package

WBS Observation

The WBS reveals that implementation is only one part of the total scope. Analysis, modelling, integration, testing, and reporting are equally necessary for successful completion.

WBS Output and Interpretation

Discussion

Work Breakdown Interpretation is an important part of the Inventory Management System because it directly affects scope control and planning discipline. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches work breakdown interpretation through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, work breakdown interpretation influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- The WBS reduces the chance of forgotten project work.
- Task granularity supports more realistic time estimation.
- Deliverables become easier to map against milestones.
- Documentation work is treated as planned scope rather than leftover work.

Observation

The discussion of work breakdown interpretation shows that the project is not only about writing code. It also requires deliberate control over scope control and planning discipline so the implemented system remains correct, understandable, and suitable for academic evaluation.

Aim and Theory of Gantt Chart

Aim

To create and analyze a Gantt-style timeline for the Inventory Management System so major tasks and dependencies can be visualized across the project schedule.

Theory

A Gantt chart maps project tasks against calendar time. It helps reviewers understand not just which tasks exist, but also when they occur and how long they remain active.

In software engineering, the Gantt view is useful because technical tasks often depend on earlier analytical work. It is therefore a practical complement to the WBS.

Gantt Schedule

Task	W1	W2	W3	W4	W5	W6	W7	W8
Requirement analysis	X	X	-	-	-	-	-	-
Design and UML	-	X	X	-	-	-	-	-
Backend development	-	-	X	X	X	-	-	-
Frontend development	-	-	-	X	X	X	-	-
Integration	-	-	-	-	X	X	-	-
Testing and documentation	-	-	-	-	-	X	X	X

Milestones and Dependencies

Discussion

Schedule Dependencies is an important part of the Inventory Management System because it directly affects ordering of work and delivery rhythm. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches schedule dependencies through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, schedule dependencies influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Requirement clarity must precede design quality.
- Design maturity improves backend and frontend alignment.
- Integration depends on both sides of implementation being ready.
- Documentation becomes stronger when created alongside project progress.

Observation

The discussion of schedule dependencies shows that the project is not only about writing code. It also requires deliberate control over ordering of work and delivery rhythm so the implemented system remains correct, understandable, and suitable for academic evaluation.

Schedule Risk and Control

Discussion

Schedule Risk is an important part of the Inventory Management System because it directly affects protecting the project timeline. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches schedule risk through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, schedule risk influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Database setup can delay full end-to-end verification.
- Late documentation leads to rushed formatting and weak explanation.
- Underestimated UI refinement can affect integration timing.
- Missing early scope control increases downstream rework.

Observation

The discussion of schedule risk shows that the project is not only about writing code. It also requires deliberate control over protecting the project timeline so the implemented system remains correct, understandable, and suitable for academic evaluation.

Aim and Theory of Software Metrics

Aim

To study the role of software metrics in measuring project size, effort, time, productivity, and quality for the Inventory Management System.

Theory

Software metrics turn qualitative judgment into manageable quantitative estimates. They allow project managers to reason about size, effort, productivity, and quality more systematically.

Metrics do not replace engineering judgment, but they support better planning and better comparison between estimated and actual work.

Size Metrics

Metric	Estimated Value	Meaning
Lines of Code	Approximately 4,500-5,000	Combined frontend, backend, and supporting assets
Function Points	Approximately 52	Feature-level estimate across auth, inventory, dashboard, and report support
Collection Count	4 primary entities	User, Category, Product, StockMovement

Effort and Time Metrics

Metric	Estimated Value	Interpretation
Effort	180 person-hours	Total projected labor across the lifecycle
Schedule	8 weeks	Calendar time for the academic project
Average task block	1 to 2 weeks	Typical milestone span for grouped activities

Metric Note

Effort and schedule must be read together because a short schedule can still contain high labor concentration.

Productivity and Quality Metrics

Metric	Estimated Value	Interpretation
Productivity	25-30 LOC per person-day	Moderate rate for structured academic work
Target response time	Below 2 seconds	Supports interactive usability
Defect density target	Below 0.8 defects/KLOC	Emphasizes correctness
Rework target	Below 15 percent	Indicates stable requirements

Metric Interpretation and Output

Discussion

Metric Interpretation is an important part of the Inventory Management System because it directly affects using numbers responsibly in project review. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches metric interpretation through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, metric interpretation influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Metrics help comparison, not blind decision making.
- Quality metrics matter more than raw code volume in inventory software.
- Productivity must be balanced against correctness and maintainability.
- Estimates should be revisited after integration and testing.

Observation

The discussion of metric interpretation shows that the project is not only about writing code. It also requires deliberate control over using numbers responsibly in project review so the implemented system remains correct, understandable, and suitable for academic evaluation.

Aim and Theory of Cost Estimation

Aim

To estimate and analyze the financial requirements of the Inventory Management System including development, testing, and maintenance cost components.

Theory

Cost estimation translates project effort into financial understanding. It supports budgeting, feasibility review, and prioritization of work.

Even for academic projects, a realistic estimate shows management maturity because it connects features and effort with resource consumption.

Development Cost Estimation

Cost Head	Amount (INR)	Reason
Requirement analysis	18,000	Scope study and requirement documentation
Design and modelling	12,000	Architecture and UML preparation
Backend development	42,000	Controllers, routes, models, authentication
Frontend development	38,000	Screens, forms, state handling, responsive UI

Testing, Documentation, and Support Cost

Cost Head	Amount (INR)	Reason
Testing and QA	14,000	Scenario validation and issue review
Documentation	10,000	SPM formatting and report output
Infrastructure and tool support	6,000	Connectivity and platform support
Maintenance reserve	12,000	Post-delivery fixes and small changes

Total Cost and Cost Observation

Total Cost

The overall estimated project cost is approximately INR 160,000 including reserve and support categories.

Observation

The estimate indicates a moderate-cost prototype-scale software project. The largest share belongs to implementation effort because coding and integration consume the majority of project labor.

Reserve categories are included intentionally because environment setup, bug fixing, and formatting changes frequently appear near delivery time.

Aim and Theory of Resource Allocation

Aim

To identify and allocate the human, hardware, software, and time resources required for the Inventory Management System.

Theory

Resource allocation helps the project move from intention to execution. A task cannot be completed unless the right capability and environment are available at the right time.

In software work, resource planning includes people, technical tools, access conditions, and schedule availability.

Human Resource Allocation

Role	Responsibility	Reason for Allocation
Project Manager	Planning and coordination	Maintains scope and schedule control
Backend Developer	API and logic implementation	Builds protected business operations
Frontend Developer	UI and interaction implementation	Creates user-facing workflow
Tester	Validation and quality review	Protects reliability and acceptance

Hardware and Software Allocation

Resource Type	Allocated Resource	Purpose
Hardware	Laptop/Desktop workstation	Coding, testing, and report work
Hardware	Internet connection	Dependencies and deployment tasks
Software	Node.js and npm	Runtime and package management
Software	MongoDB Atlas	Persistent data layer
Software	Browser and code editor	Testing and development

Time Resource Allocation and Observation

Discussion

Time Allocation is an important part of the Inventory Management System because it directly affects balancing analysis, development, and documentation. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches time allocation through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, time allocation influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Early weeks focus on analysis and design.
- Middle weeks focus on backend and frontend implementation.
- Final weeks focus on integration, testing, deployment, and report completion.
- Documentation is strongest when written incrementally rather than postponed.

Observation

The discussion of time allocation shows that the project is not only about writing code. It also requires deliberate control over balancing analysis, development, and documentation so the implemented system remains correct, understandable, and suitable for academic evaluation.

Aim and Theory of Team Structure

Aim

To study and design the team structure, reporting relationships, and role responsibilities suitable for the Inventory Management System.

Theory

Team structure makes ownership visible. It shows who plans, who implements, who verifies, and who closes the project.

Without team structure, software projects often suffer from overlap, missing accountability, and unclear communication.

Team Roles and Responsibilities

Role	Main Duties	Primary Deliverable
Project Manager	Coordinate work and monitor progress	Controlled timeline
Backend Developer	Build data and API logic	Stable server-side implementation
Frontend Developer	Build user interaction and layout	Usable client-side implementation
Tester	Review scenarios and report faults	Quality findings

Responsibility Matrix

Activity	Project Manager	Backend Dev	Frontend Dev	Tester
Requirement review	High	Medium	Medium	Low
API implementation	Low	High	Low	Medium
UI implementation	Low	Low	High	Medium
Testing	Medium	Medium	Medium	High
Documentation closure	High	Medium	Medium	Low

Communication and Governance Plan

Discussion

Communication Plan is an important part of the Inventory Management System because it directly affects coordination and issue escalation. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches communication plan through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, communication plan influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Weekly review helps compare planned and completed work.
- Integration issues should be discussed quickly rather than accumulated.
- Documentation decisions should follow implementation reality.
- Final submission should be made only from verified files.

Observation

The discussion of communication plan shows that the project is not only about writing code. It also requires deliberate control over coordination and issue escalation so the implemented system remains correct, understandable, and suitable for academic evaluation.

Testing Strategy

Discussion

Testing Strategy is an important part of the Inventory Management System because it directly affects verifying behavior rather than only appearance. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches testing strategy through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, testing strategy influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Authentication flow must accept valid users and reject invalid input.
- Product operations must preserve unique SKU behavior.
- Stock-out must never exceed available quantity.
- Responsive layouts must remain readable on smaller screens.

Observation

The discussion of testing strategy shows that the project is not only about writing code. It also requires deliberate control over verifying behavior rather than only appearance so the implemented system remains correct, understandable, and suitable for academic evaluation.

Representative Test Cases

Test ID	Scenario	Expected Result
TC-01	Login with valid credentials	Access granted and dashboard loaded
TC-02	Create duplicate SKU	Validation error shown
TC-03	Record invalid stock-out	Operation rejected
TC-04	Update product quantity	Status recalculated correctly
TC-05	Load on narrow screen	Layout remains usable

Validation and Error Handling

Discussion

Validation Strategy is an important part of the Inventory Management System because it directly affects protecting the system from inconsistent state. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches validation strategy through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, validation strategy influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Required fields reduce incomplete record creation.
- Numeric conversion ensures quantity and price consistency.
- Error middleware standardizes backend failure responses.
- Frontend feedback panels help users correct problems quickly.

Observation

The discussion of validation strategy shows that the project is not only about writing code. It also requires deliberate control over protecting the system from inconsistent state so the implemented system remains correct, understandable, and suitable for academic evaluation.

Risk Management

Discussion

Risk Management is an important part of the Inventory Management System because it directly affects anticipating technical and process-level disruption. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches risk management through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, risk management influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Database configuration may fail or be delayed.
- Deployment settings may be entered incorrectly.
- Late report work may reduce documentation quality.
- Requirement drift may create unnecessary implementation churn.

Observation

The discussion of risk management shows that the project is not only about writing code. It also requires deliberate control over anticipating technical and process-level disruption so the implemented system remains correct, understandable, and suitable for academic evaluation.

Feasibility Analysis

Feasibility Type	Assessment
Technical	High due to mainstream stack and modular design
Operational	High because workflows are simple and task-oriented
Economic	Reasonable for prototype and academic scale
Schedule	Achievable with disciplined planning

Implementation Summary

Area	Location	Status
Backend	backend/src	Implemented
Frontend	frontend/src	Implemented
Deployment Config	render.yaml	Prepared
Documentation	docs	Prepared and expandable

Summary

The delivered project includes the application code, deployment configuration, and report artefacts. It is therefore complete as an academic deliverable and ready for further verification and extension.

Deployment Notes

Discussion

Deployment Preparation is an important part of the Inventory Management System because it directly affects production readiness and environment setup. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches deployment preparation through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, deployment preparation influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Render configuration has been added to the project.
- MongoDB Atlas connection is configured through environment variables.
- The backend can serve the built frontend for single-service deployment.
- Health check and production scripts are defined for deployment use.

Observation

The discussion of deployment preparation shows that the project is not only about writing code. It also requires deliberate control over production readiness and environment setup so the implemented system remains correct, understandable, and suitable for academic evaluation.

Future Enhancements

Discussion

Future Enhancement Planning is an important part of the Inventory Management System because it directly affects extending the system beyond the current scope. In an academic software project, this area must be documented clearly so the reviewer can understand why the chosen design and planning decisions are reasonable.

The project approaches future enhancement planning through a practical MERN perspective. Instead of treating the subject as an isolated theory topic, the report connects it with inventory tasks such as product registration, quantity tracking, stock validation, dashboard reporting, and role-based usage.

From a management angle, future enhancement planning influences scope stability, implementation order, testing effort, and maintenance readiness. Therefore it deserves dedicated space in the report rather than being merged into a short summary.

Key Points

- Barcode support could speed up movement entry.
- Purchase-order workflow could improve replenishment tracking.
- Export features could support reporting and audit needs.
- Manager-specific analytics could extend the role model.

Observation

The discussion of future enhancement planning shows that the project is not only about writing code. It also requires deliberate control over extending the system beyond the current scope so the implemented system remains correct, understandable, and suitable for academic evaluation.

Conclusion

Conclusion

The Inventory Management System demonstrates a complete cycle from requirement understanding to deployable implementation. The project combines practical inventory features with structured software project management thinking.

Through SRS preparation, UML modelling, WBS and Gantt planning, metric analysis, cost estimation, resource allocation, team organization, testing, and deployment preparation, the report shows that successful software development depends on both technical execution and managerial discipline.

The final deliverable therefore serves both as a working MERN application and as a comprehensive academic report comparable in style and range to the reference sample.

References

Reference Basis

- Standard software project management concepts for planning, control, and quality assurance.
- Common SRS organization patterns inspired by formal software engineering documentation practice.
- UML conventions for use case, class, activity, sequence, component, and deployment representation.
- The implemented Inventory Management System source files included in the project workspace.